Lab 5: Static Code Analysis – Bank Account System

Nov 2, 2026 · @rhea

Overview

After the fixes, `bank_account_system.py` scores 10.00/10 on Pylint (up from 5.62), and Bandit and Flake8 report 0 issues. The program still runs, and it now gives correct balances.

Item	Detail
Student	Rhea Menon (PES1UG24AM222)
Assigned repo	SETAPESU26/L5_40_bank_account_systen
File analysed	`bank_account_system.py`
Tools	Pylint 4.1.3, Bandit 1.9.4, Flake8 7.5.0 (Python 3.13)
Setup	`python3 -m venv venv`, `source venv/bin/activate`, `pip install pylint bandit flake8`

The reports were generated with the handout's commands: `pylint bank_account_system.py > pylint_report.txt`, `bandit -r bank_account_system.py > bandit_report.txt` and `flake8 bank_account_system.py > flake8_report.txt`. Before and after copies are in `reports/before/` and `reports/after/` in the repo.

Program and initial run

The original program ran without errors, but its output showed two problems. The program is a bank ledger: it creates account A01 with 1000, deposits 500, withdraws 200, computes a 5% bonus, hashes a PIN, generates a temporary PIN and writes a backup file.

```
Bonus: 65.0
PIN Hash: 81dc9bdb52d04dc20036dbd8313ed055
Temp PIN: 4566
Backup complete: backup.txt
Token initialized: SUPER_SECRET_BANK_TOKEN_2026
```

- **Wrong balance:** a bonus of 65.0 means the balance was 1300. `withdraw` defines `service_fee = 2.50` but never charges it, so the balance should have been 1297.50.
- **Secret leaked:** the admin token is hard-coded and printed in plain text.
- **Weak PIN hash:** the hash is the well-known unsalted MD5 of "1234".

Before results

The baseline scan found 34 messages across the three tools, including two High-severity security issues.

Tool	Result before fixes	Main findings
Pylint	5.62 / 10 (21 messages)	`eval`, mutable default, bare except, unused variables, missing docstrings, no `with` on `open`
Bandit	6 issues: 2 High, 1 Medium, 3 Low	B605 shell call, B324 MD5, B307 `eval`, B110 try/except/pass, B311 `random`, B105 hard-coded token
Flake8	7 issues	F401, E501, E722, E711, F841 (×2), W292

Issue table

All 17 documented issues were fixed. They are ordered high severity first, and line numbers refer to the original file.

#	Issue	Type	Tool / code	Severity	Line(s)	Description	Fix approach
1	Shell command injection	Security	Bandit B605	High	55	`os.system("echo ... " + filename)` builds a shell command from a parameter, so a crafted filename runs arbitrary commands.	Removed the shell call and used `print()`.
2	Weak MD5 PIN hash	Security	Bandit B324	High	44	Unsalted MD5 is fast and broken, so PINs can be brute-forced instantly.	Salted PBKDF2-HMAC-SHA256 (100,000 iterations), returned as `salt$digest`.
3	Use of `eval()`	Security	Bandit B307, Pylint W0123	Medium	39	Evaluates a caller-supplied string, so any Python code can be executed.	Takes a numeric `rate` and returns `balance * float(rate)`.

#	Issue	Type	Tool / code	Severity	Line(s)	Description	Fix approach
4	Mutable default argument	Bug	Pylint W0102	Medium	11	`transactions=[]` is shared across calls, so logs from different accounts mix.	Default is `None`; a new list is created and stored with each account.
5	Service fee never charged	Bug (logic)	Flake8 F841, Pylint W0612	Medium	27–34	`service_fee` was unused, so withdrawals were free (balance 1300, not 1297.50).	Check and deduct `amount + SERVICE_FEE`.
6	Bare `except:` with `pass`	Bug	Bandit B110, Pylint W0702, Flake8 E722	Low	20–23	All errors were silently swallowed, so a deposit to a missing account looked successful.	Catch `KeyError`, return True/False, reject non-positive amounts.
7	Hard-coded, printed token	Security	Bandit B105	Low	59, 67	Secret stored in source and printed to the console.	Read from the `BANK_ADMIN_TOKEN` env var; print only whether it is set.
8	Insecure random PIN	Security	Bandit B311	Low	48	`random` is predictable.	`1000 + secrets.randbelow(9000)`
9	`open` without `with` / encoding	Quality	Pylint R1732, W1514	Low	52–54	File left open on error; encoding depends on the platform.	`with open(..., encoding="utf-8")`
10	Non-snake_case name	Style	Pylint C0103	Low	11	`createAccount` breaks PEP 8 naming.	Renamed to `create_account`.
11	`%d` formatting	Bug / style	Pylint C0209	Low	15	`%d` truncates float balances in the log.	f-string with `:.2f`.
12	`== None`	Style	Flake8 E711, Pylint C0121	Low	28	Equality check with `None`.	`is None`
13	`else` after `return`	Style	Pylint R1705	Low	32	Unnecessary nesting.	Removed `else`.
14	Missing docstrings	Style	Pylint C0116	Low	11–58	7 functions had no docstring.	Docstrings added.
15	Unused import / variable	Style	Flake8 F401, F841; Pylint W0611, W0612	Low	6, 38	`sys` was unused; `bal` existed only for `eval`.	Removed `sys`; `balance` is now used directly.
16	Long line, no final newline	Style	Flake8 E501, W292; Pylint C0304	Low	15, 70	Line was 89 characters (limit 79); no newline at end of file.	Wrapped the line; added a trailing newline.
17	`main()` runs on import	Quality	Manual review	Low	70	Importing the module runs the demo and writes a file.	`if __name__ == "__main__":` guard.

Commits

There is one commit per tool, plus one commit for the baseline reports and one for the documentation.

Order	Commit message	What changed	Result after commit
1	Add baseline Pylint, Bandit and Flake8 reports	`reports/before/*.txt`	Baseline recorded
2	**Fix all security issues reported by Bandit**	Removed `eval`, MD5, `os.system`, `random` and the hard-coded token; replaced `try/except/pass` with `except KeyError`	Bandit 0 issues
3	**Fix all style and lint issues reported by Flake8**	Removed `sys`; wrapped the long line; `is None`; charged the service fee; added the final newline	Flake8 0 issues
4	**Fix code quality issues reported by Pylint (5.62 -> 10.00/10)**	Docstrings, `create_account`, mutable default, f-string, `with open`, constants, amount validation, `__main__` guard	Pylint 10.00/10
5	Add after reports, issue table and reflection	`reports/after/*.txt`, `issues.md`, `reflection.md`	Final reports recorded

The three tool commits are 2–4. A few findings were reported by more than one tool (for example, bare `except` by Bandit, Pylint and Flake8). Each was fixed once, in the first commit that touched it.

After results

Every issue found by the three tools is resolved, and the program output is now correct.

Tool	Before	After
Pylint	5.62 / 10	**10.00 / 10**
Bandit	6 issues (2 High, 1 Medium, 3 Low)	**0 issues**
Flake8	7 issues	**0 issues**

Final program output (the hash and temp PIN change on every run):

```
Balance: 1297.5
Bonus: 64.875
PIN Hash: 1b413b10e1cd8e88c071f8a69a32fab9$af5b02aba2624dc3...
Temp PIN: 7668
Backup complete: backup.txt
Token initialized: False
```

The balance now includes the 2.50 withdrawal fee, and the token is no longer printed. Extra checks also passed: duplicate accounts and negative or zero deposits are rejected, each account keeps its own transaction log, and a PIN hash can be verified using its stored salt.

Reflection

1. Which issues were the easiest to fix, and which were the hardest? Why?

The easiest were the Flake8 and Pylint style issues: the unused import, `== None`, the final newline, the long line, the snake_case rename and the docstrings. Each one points to an exact line and rule, and the fix does not change behaviour.

The hardest needed a design decision, not a one-line change. Removing `eval()` meant changing `compute_bonus` to take a numeric rate and updating its caller. Replacing MD5 properly meant adding a salt and a slow hash (PBKDF2), not just switching to SHA-256. The unused `service_fee` was only reported as an unused variable, but the real problem was that withdrawals were never charged. Deleting the line would have silenced the warning and hidden the bug.

2. Did the static analysis tools report any false positives? If so, describe one example.

There were no outright false positives here, but some findings needed judgement. Bandit B105 (hard-coded password) guesses from the variable name, so it reports only Medium confidence. It would also fire on a harmless constant like `TOKEN_HEADER = "Authorization"`. Bandit B311 is a false positive whenever `random` is used for non-security work such as shuffling demo data; here it generated a PIN, so it was valid. Several issues were reported by two or three tools at once, so the raw count overstates how many distinct problems there were.

3. How would you integrate static analysis tools into your actual software development workflow?

- Run Flake8 and Pylint in the editor so issues show up while coding.
- Add a pre-commit hook that runs all three tools, so bad code cannot be committed.
- Add a CI job (e.g. GitHub Actions) on every push and pull request. It fails on any Medium/High Bandit issue or any Flake8 issue, and runs `pylint --fail-under=9.5`.

- Keep shared config files (`.pylintrc` , `setup.cfg`) in the repo, and require that any `# nosec` or `# pylint: disable` is justified in code review.

4. What tangible improvements did you observe in code quality, readability, or potential robustness after applying the fixes?

- **Security:** there is no shell execution, `eval` , MD5, hard-coded secret or predictable PIN generator left.
- **Correctness:** the fee is charged (balance 1297.50 instead of 1300). Each account has its own transaction log. Invalid deposits are rejected instead of silently ignored.
- **Robustness:** the backup file is always closed and written as UTF-8. Importing the module has no side effects.
- **Readability:** every function has a docstring, names follow PEP 8 and magic numbers are named constants. Pylint went from 5.62 to 10.00/10.